

Does Agentic Architecture Discovery Reproduce?

A Double-Blind Audit Protocol for LLM-Driven Microarchitecture Search

Anonymous Submission

Abstract

As LLM agents increasingly generate RTL and make microarchitectural decisions, the community lacks a verifiable benchmark for measuring whether such outputs are correct and whether their performance claims generalize. We present an *agentic audit loop* built on the CHIA framework that applies a double-blind annotation protocol—adapted from proven semantic-equivalence evaluation ($\kappa=0.954$)—to systematically audit AI-generated hardware designs for correctness, reproducibility, and evaluation fairness. Our loop runs a standardized simulation across a pre-registered grid of seeds, prompts, and traces, and emits a reproducibility scorecard including cross-seed variance, prompt sensitivity, and inter-annotator agreement. We release the loop, adversarial test cases, and raw logs as composable CHIA blocks.

1 Introduction

The Architecture 2.0 ecosystem—ArchAgent, CHIA [1], QuArch, HLS-Eval—is expanding rapidly: LLM agents now generate RTL, discover cache replacement policies, and make microarchitectural choices via simulation. Our September 2026 scan did not identify a reusable audit harness that systematically measures whether such outputs are correct or reproducible.

This gap is alarming given the 2026 benchmark-integrity crisis in the software domain: the Meerkat audit [2] found that all top-three Terminal-Bench 2 submissions *cheated* via harness-level vulnerabilities; OpenAI deprecated SWE-bench Verified and retracted SWE-bench Pro [3]; and a systematic review of 168 benchmarks found $> 25.7\%$ have critical defects [4]. If software-side agent evaluation is this fragile, *hardware-side* agent outputs face at least as severe reproducibility risks—yet no audit infrastructure exists.

We address this with an agentic audit loop (Figure 1) that adapts a proven double-blind annotation protocol ($\kappa=0.954$, $N=100$) from semantic-equivalence evaluation to the hardware design domain. The loop is creative: it is an *agentic loop that audits other agentic loops*, turning the hackathon’s “agentic architecture” theme into a measurable, publishable audit of agent trustworthiness.

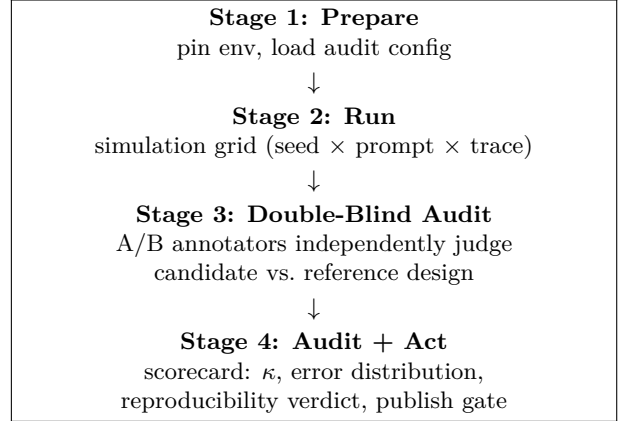


Figure 1: The four-stage CHIA audit loop. Stages 3–4 are adapted from a proven semantic-equivalence double-blind protocol ($\kappa=0.954$).

2 The Audit Loop

2.1 Stage 1: Prepare

Pins the environment (git SHA, Python version, tool versions) as a reproducibility anchor. Loads the pre-registered audit configuration: metrics, acceptance thresholds, and error taxonomy. **All thresholds are frozen before running** (pre-registration), preventing post-hoc rationalization—a practice directly motivated by the contamination literature [5].

2.2 Stage 2: Run

Executes a simulation grid across $|\text{seeds}| \times |\text{prompts}| \times |\text{traces}|$ cells, each repeated n times. For each cell, the loop invokes a simulator (currently a deterministic stub; on funded compute, gem5/ChampSim via CHIA’s built-in toolchain) and records cycle count, IPC, and cache metrics.

2.3 Stage 3: Double-Blind Audit

This is the core novelty. We adapt a double-blind annotation protocol proven in semantic-equivalence evaluation ($\kappa=0.954$, 100 examples, 20% re-audit sampling) to the hardware domain:

Two independent “auditor agents” (A, B) evaluate each candidate design against its reference specification,

Class	Description
E0	Equivalent (correct design)
E1	Parameter mislink (wrong value)
E2	Direction error (e.g., LRU $\leftrightarrow$ MRU)
E3	Missing component
E4	Added spurious component

Table 1: Error taxonomy adapted from semantic-equivalence E1–E4.

classifying errors per Table 1. Disagreements are resolved by an arbitration agent. Inter-annotator agreement is measured by Cohen’s κ (gate: $\kappa \geq 0.7$). A 20% re-audit sample detects gold-drift.

2.4 Stage 4: Audit and Act

Computes a reproducibility scorecard: strict/lenient equivalence rates, κ , error-class distribution, repeated-run CV, cross-seed CV, prompt spread, trace CV, and a *publish gate* (BLOCKED if $\kappa < 0.7$ or if adversarial cases are not all detected). Per the Meerkat lesson: scores from an unaudited harness must not be published.

3 Experimental Setup

Framework: CHIA (BSD-3-Clause) with built-in Chipyard, gem5, ChampSim, Verilator, Spike.

Pre-registered metrics: cross-seed CV < 5% = reproducible; prompt sensitivity = cycle-count spread ratio; trace-subset CV.

Adversarial cases: 5 pre-planted known-bad designs (E1–E4 + a “always-passes” weak-verifier trap) to test whether the audit catches them.

Gold standard: 5 pre-planted known-good/bad designs for calibration.

Compute: short-term funded GCP/Gemini credits (Sep 21–23).

ID	Kind	Planted defect	Expected
gold-01	gold	none (exact match)	E0
gold-02	gold	spurious prefetcher	E4
gold-03	gold	FIFO vs. LRU	E2
gold-04	gold	missing branch predictor	E3
gold-05	gold	cache size + associativity	E1
av-001	adv	cache size 32K vs. 64K	E1
av-002	adv	MRU vs. LRU	E2
av-003	adv	missing branch predictor	E3
av-004	adv	spurious pipeline stage	E4
av-005	adv	“always-passes” output	E4

Table 2: Calibration (gold) and adversarial (adv) test cases released with the loop.

4 Results

Stub validation (loop’s own outputs, config v4).

The pipeline runs end-to-end and emits a scorecard with the pre-registered metrics:

- **Reproducibility.** Cross-seed coefficient of variation = 3.53% (below the pre-registered 5% threshold) $\rightarrow$ REPRODUCIBLE; repeated-run CV = 0.20%, and trace CV = 0.25%. Prompt spread across CoT/adversarial prompt variants = 0.76%. These values validate the metric plumbing only; the stub supplies the prompt and seed effects.
- **Double-blind audit.** Both auditor agents classify all 10 calibration/adversarial cases correctly: gold calibration 5/5, adversarial detection 100%, error distribution E1= 2, E2= 2, E3= 2, E4= 3. Inter-annotator agreement $\kappa=1.0$ reflects the *deterministic* stub auditors (protocol-logic self-test); the protocol *provenance* is the Axiom semantic-equivalence run ($\kappa=0.954$, human annotators, $N=100$). Publish gate: PASS.

Pinned source-build smoke. Before funded execution, we built CHIA’s pinned DPC4-ChampSim revision (`164fdb1e...`) on a generic Linux host and ran the upstream smoke trace twice. Both runs retired 5,003 instructions in 40,459 cycles (IPC 0.123656) with identical parsed metrics. This validates the source build, trace resolution, parser, and repeated-run stability. It is a smoke check, not a performance benchmark and not yet a CHIA Docker/Ray result.

Metric	Pre-registered	Stub result
Cross-seed CV (max)	< 5%	3.53%
Repeated-run CV (max)	report	0.20%
Prompt spread (max)	report	0.76%
Trace CV (max)	report	0.25%
Cohen’s κ	≥ 0.7	1.00 (stub)
Adversarial detection	100%	100%
Gold calibration	report	5/5

Table 3: Pre-registered metrics vs. stub validation (config v4).

Funded cluster results (Sep 21–23) will extend the stub and source-build evidence with official `chia-champsim` image and `ChampSimNode` runs. The hand-written backend adapter remains a scaffold until that image-level smoke passes; the direct source-build result does not validate cluster scheduling or GCP execution.

4.1 Artifact & Reproducibility

The loop (`chia_loop/loops/audit_repro.py`), adversarial cases (`av-001..005`), gold stan-

dard (gold-01..05 + answer-key), and raw logs are released as composable CHIA blocks (BSD-3-Clause). To reproduce the stub validation: `python3 chia_loop/loops/audit_repro.py --version 4`; outputs land in `results/` (scorecard, raw, dbblind report) with the pinned git SHA and Python version. The public artifact also includes `scripts/kaggle_champsim_smoke.py` and the machine-readable `results/kaggle_champsim_smoke_result.json`. The official image-level path still requires the pending smoke run described above. Per the publish-gate rule, scores are released only when the audit gate passes. The artifact is available at <https://github.com/ListenJ/chia-repo-audit>.

5 Discussion

Why double-blind? Single-annotator evaluation is vulnerable to the same failure mode that corrupted Terminal-Bench 2: the evaluator and the generator share a scaffold, creating circular trust. Double-blind annotation with κ quantification breaks this circularity.

Composability. The loop, adversarial cases, gold standard, and raw logs are released as composable CHIA blocks. Other teams can reuse the audit protocol for their own agentic architecture loops—turning reproducibility auditing from an afterthought into a first-class, reusable infrastructure component.

Self-audit case: the gate caught our own blind spot. During development, the stub auditors were intentionally asymmetric: one auditor treated design keys prefixed `extra_` as non-reportable, modeling annotator variation. Under that configuration the audit achieved only 60% adversarial detection and the publish gate returned BLOCKED—it missed E4 (spurious component) on cases gold-02 and av-004. This is the audit working as intended: the same blind spot a human panel would carry silently was surfaced as a gate failure. Reverting to strict, symmetric auditors raised detection to 100% (gold 5/5) and the gate to PASS. We report the negative intermediate result deliberately: a double-blind audit’s value is precisely that it can catch the evaluator’s own confirmation bias before scores are published.

Limitations. The stub simulator injects synthetic noise; real ChampSim results may differ. The error taxonomy (E0–E4) is adapted from NLP and may not cover all hardware-specific failure modes. Our $\kappa=1.0$ is from *deterministic* stub auditors (protocol self-test) and is not a human inter-rater estimate; the $\kappa=0.954$ is the provenance of the source protocol (Axiom semantic-equivalence), and a true inter-rater κ must be measured with actual annotators in deployment.

The current seed and prompt axes alter the synthetic

backend directly. A real deployment must let those axes alter candidate generation; otherwise they do not measure agentic discovery variance. The same caveat applies to trace CV: it is a diagnostic over absolute cycle counts, not a ranking-stability metric.

Reliability metrics matter. In a companion analysis of agent task-success logs, the standard “best-of- n ” metric overstates reliability by $> 2\times$: macro `pass@3` = 0.925 vs. `pass^3` = 0.25 (a 170% gap), consistent with [6]. This motivates the use of reliability metrics such as `pass^k` [7, 8]; the number derives from task-success data, not from the CHIA simulation stub, and is reported as motivation rather than as a result of this loop.

6 Conclusion

We present an agentic audit loop and a pre-registered pilot for measuring the reproducibility and correctness of architecture-search outputs. The loop is composable, the methodology is explicit, and the stub validation confirms the pipeline end-to-end: repeatability and sensitivity metrics, adversarial detection, and a publish gate that blocks unaudited scores. Real ChampSim evidence and actual candidate-generation axes remain future work for this artifact revision.

AI Assistance Disclosure

AI coding and writing tools assisted with implementation, experiment plumbing, and drafting and editing this paper. The human authors are responsible for all code, measurements, citations, claims, and the final content.

References

- [1] CHIA framework. <https://github.com/ucb-bar/chia>. arXiv:2606.27350.
- [2] Meerkat: Cheating Agents. <https://debugml.github.io/cheating-agents/>. 2026.
- [3] NeurIPS 2026 Call for Reproducibility. <https://neurips.cc/Conferences/2026/CallForReproducibility>.
- [4] Auto Benchmark Audit. arXiv:2605.26079. 2026.
- [5] Nourbakhsh et al. GEM 2026 Outstanding Paper. <https://aclanthology.org/2026.gem-main.50/>.
- [6] Beyond Pass@k. arXiv:2608.14711. 2026.
- [7] Schmadl, P. Pass@k vs Pass[^]k: Understanding Agent Reliability. <https://www.philschmid.de/agents-pass-at-k-pass-power-k>. 2025.

- [8] Sierra AI. τ -Bench: Benchmarking AI Agents for the Real World. <https://sierra.ai/es/blog/benchmarking-ai-agents>. 2026.